

# Microservices Decomposition Strategies

## Decomposition of Microservices Architectures

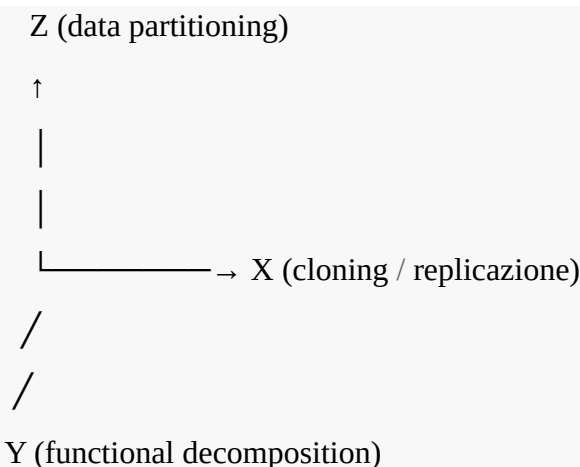
### La motivazione tecnica principale

*"Sebbene il deployment indipendente e l'autonomia dei team siano vantaggi enormi, la ragione principale [della decomposizione] è consentire alle diverse parti dell'applicazione di scalare in modo indipendente in base alle loro specifiche esigenze. Questo perché non tutte le parti dell'applicazione hanno gli stessi requisiti di scalabilità."*

### ★ Lo Scale Cube (dal libro *The Art of Scalability*)

*"Quando si parla di scalabilità, molti pensano immediatamente solo a scalabilità orizzontale o verticale. Tuttavia il libro **The Art of Scalability** presenta un modello molto utile chiamato **Scale Cube**, che descrive **tre assi distinti** per scalare un'applicazione."*

**Definizione: Scale Cube** — un modello tridimensionale in cui ogni asse rappresenta una strategia di scalabilità diversa e indipendente dalle altre.



| Asse | Nome                             | Cosa fai                                                                                                                        | Esempio                                                 | Dove lo abbiamo visto |
|------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|-----------------------|
| X    | Horizontal Duplication / Cloning | Esegui <b>più copie identiche</b> dell'intera applicazione dietro un <b>load balancer</b> , distribuendo le richieste           | WithReplicas(3)                                         | Lezione 4             |
| Y    | Functional Decomposition<br>★    | <b>Scomponi l'applicazione per funzione o capability di business</b> in servizi più piccoli e indipendenti, ognuno responsabile | Servizi separati per Catalog, Basket, Ordering, Payment | Lezione 5 e 6         |

| Asse | Nome                                | Cosa fai                                                                                                                    | Esempio                                           | Dove lo abbiamo visto |
|------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|-----------------------|
|      |                                     | di un insieme specifico di attività correlate                                                                               |                                                   |                       |
| Z    | <b>Data Partitioning / Sharding</b> | Esegui <b>più istanze identiche di un servizio</b> , ma <b>ogni istanza è responsabile solo di un sottoinsieme dei dati</b> | Clienti EU su un'istanza, clienti USA su un'altra | <b>Lezione 10</b>     |

**Chiarimenti importanti su ciascun asse**

#### Asse X — la clonazione

*"Si eseguono più copie identiche dell'intera applicazione o dei microservizi dietro un load balancer. **Abbiamo già applicato questo concetto** al nostro API Service monolitico nelle sezioni precedenti. È un metodo comune ed efficace."*

Il limite: scali **tutto insieme**, anche ciò che non serve.

#### Asse Y — la decomposizione funzionale ★

*"Questa è **la dimensione più direttamente correlata all'architettura a microservizi**, piuttosto che la semplice clonazione dell'intera applicazione. Quando un'applicazione coesa viene scomposta in queste unità più piccole, indipendenti e deployabili, **si sta applicando lo scaling sull'asse Y.**"*

*"Invece di un'unica grande applicazione, si hanno **servizi separati per prodotti, carrello, ordini e pagamenti.**"*

#### Asse Z — lo sharding

*"Simile all'asse X: si eseguono più istanze identiche di un servizio, **ma ogni istanza è responsabile solo di un sottoinsieme dei dati.** Ad esempio, si possono partizionare i dati dei clienti **per area geografica**, con istanze diverse che gestiscono clienti di regioni diverse."*

*"Questo approccio si usa spesso per **scalare o partizionare database con dataset molto grandi**, o per distribuire il carico in base ad attributi specifici dei dati."*

#### Il focus di questa lezione

*"Nella nostra attuale discussione ci concentreremo principalmente sullo **scaling sull'asse Y, ovvero la decomposizione funzionale.** Il nostro obiettivo è suddividere il servizio API unificato in servizi più piccoli e gestibili, ciascuno focalizzato su una specifica capability di business."*

*"Comprendere l'asse Y è il primo passo. **La domanda successiva è: come definiamo i confini di questi nuovi servizi, e quali principi o pattern possono guidarci?**"*

# Decomposition Patterns: Decompose by Business Capability / by Subdomain

## I due obiettivi di ogni decomposizione

Prima dei pattern, la lezione fissa **cosa stiamo cercando di ottenere**:

### 1. Alta coesione (high cohesion)

**Definizione:** un microservizio dovrebbe **raggruppare funzioni strettamente correlate** che lavorano insieme per raggiungere un obiettivo comune.

*"In sostanza, **un servizio dovrebbe svolgere una singola funzione** — o essere responsabile di un'area funzionale molto specifica — e svolgerla bene."*

### 2. Basso accoppiamento (loose coupling)

**Definizione:** ogni servizio dovrebbe essere **il più indipendente possibile** dagli altri, con i **dettagli di implementazione nascosti** e le funzionalità esposte tramite **un'API stabile e ben definita**.

*"Le modifiche a un servizio **non dovrebbero riflettersi sugli altri**, a condizione che **il contratto API sia mantenuto**."*

**Nota concettuale:** coesione e accoppiamento sono due facce della stessa medaglia. Se metti insieme cose che non c'entrano → bassa coesione. Se separi cose che sono strettamente legate → alto accoppiamento (i due servizi si chiameranno continuamente).

**Il confine giusto è quello che massimizza la coesione interna e minimizza l'accoppiamento esterno.**

---

## Pattern 1 — Decompose by Business Capability

**Definizione:** identificare e definire i microservizi in base alle **capability di business principali dell'organizzazione**.

**Definizione:** **business capability** — descrive **cosa fa un'azienda** per raggiungere i propri obiettivi e generare valore. Sono tipicamente **funzioni di business stabili e di alto livello**.

### Esempi nel dominio e-commerce:

| Business Capability        | Cosa gestisce                                   |
|----------------------------|-------------------------------------------------|
| Product Catalog Management | Informazioni prodotto, categorie, prezzi        |
| Inventory Management       | Livelli di stock, prenotazioni, evasione ordini |
| Shipping Tracking          | Monitoraggio delle spedizioni                   |
| Delivery Management        | Gestione delle consegne                         |
| Payment Processing         | Elaborazione dei pagamenti                      |

### ★ Il vantaggio chiave — la stabilità:

*"Strutturando i servizi attorno a queste capability, **si creano servizi relativamente stabili**, poiché **le capability di business core tendono a cambiare meno frequentemente** rispetto a processi o tecnologie specifici."*

Questo è il punto forte del pattern: un'azienda che vende prodotti online avrà *sempre* bisogno di gestire un catalogo, di incassare pagamenti e di spedire. L'implementazione cambierà mille volte, **la capability no**.

---

## Pattern 2 — Decompose by Subdomain (DDD)

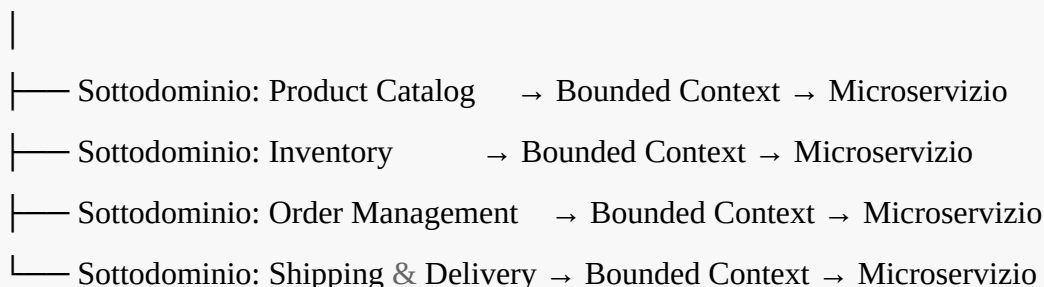
*"Un altro pattern potente e ampiamente adottato è la **decomposizione per sottodominio**, fortemente influenzata dal **Domain-Driven Design (DDD)**."*

### Definizione: Domain-Driven Design

Un approccio allo sviluppo software che si concentra sulla creazione di **una profonda comprensione dell'area problematica dell'applicazione**, che chiamiamo **dominio**.

### La gerarchia concettuale del DDD:

DOMINIO (e-commerce)



### Definizione: sottodominio (subdomain)

Una **parte distinta del business** con **la propria logica, le proprie regole e i propri dati**.

### Definizione: Bounded Context ★

*"Ciascun sottodominio ha spesso **un proprio modello concettuale e un vocabolario specifico** — quello che chiamiamo un **linguaggio coerente entro confini ben definiti**, noto come **bounded context**."*

**Perché conta il "linguaggio"**: la parola *Product* significa cose diverse in contesti diversi. Nel **Catalog** è un oggetto ricco: descrizione, immagini, categorie, specifiche. Nel **Basket** è poco più di un ID, un nome, un prezzo e una quantità. In **Shipping** è peso e dimensioni. **Il bounded context è esattamente il confine entro cui una definizione resta valida** — e quindi un candidato naturale a diventare un microservizio.

### La regola:

*"L'idea è **definire i microservizi in modo che siano allineati a questi bounded context / sottodomini**. Questo garantisce che ogni microservizio abbia **un focus ben definito, un modello interno coerente e una responsabilità chiara all'interno del dominio di business più ampio**."*

### L'avvertenza pratica:

*"Definire questi sottodomini e i loro bounded context richiede spesso **un'approfondita analisi di business e la collaborazione con esperti del dominio**."*

Non è un esercizio che si fa da soli davanti a un IDE.

**Differenza tra i due pattern:** sono complementari più che alternativi. *Business Capability* parte dall'**organigramma e dalla value chain aziendale** ("cosa fa l'azienda"). *Subdomain* parte dal **modello di dominio e dal linguaggio** ("come pensa l'azienda"). Spesso convergono sugli stessi confini — e quando non convergono, hai trovato un punto che vale la pena discutere con il business.

---

## ★ La checklist di validazione dei confini (7 domande)

*"Quando si scompone un'applicazione monolitica, è fondamentale valutare i confini dei servizi proposti."*

### 1. Single Responsibility

*"Ogni microservizio svolge effettivamente **un singolo compito**, o si concentra su **un insieme altamente coeso di funzioni correlate**?"*

### 2. Dimensione del servizio ★

*"Il servizio è **sufficientemente piccolo** da poter essere sviluppato e gestito da **un piccolo team indipendente** — ad esempio un team di due persone? Al contrario, è **troppo piccolo**, con conseguente **proliferazione di microservizi che creano un carico di comunicazione eccessivo**?"*

Nota che il criterio di dimensione è **organizzativo** (un piccolo team può gestirlo?) non tecnico (quante righe di codice ha?).

### 3. Pattern di comunicazione ★ — la domanda più diagnostica

*"I servizi proposti **si chiamano continuamente a vicenda** per eseguire operazioni di base? **Se due servizi comunicano eccessivamente, potrebbe indicare che i loro confini non sono definiti correttamente, e forse dovrebbero essere uniti in un unico servizio per ridurre overhead e congestione di rete.**"*

Questo è il test più utile in pratica: **la "chattiness" è il sintomo numero uno di un confine sbagliato**. E la soluzione proposta è controintuitiva ma corretta: **unire, non separare ulteriormente**.

### 4. Proprietà e isolamento dei dati

*"Ogni servizio è **chiaramente responsabile dei propri dati**? Il database segue il pattern **Database-per-Service**? **Come vengono gestiti i dati di cui più servizi potrebbero aver bisogno** — tramite **replica dei dati, aggiornamenti basati su eventi, o chiamate API**?"*

Le tre strategie citate anticipano le Lezioni 9 (eventi), 11 (replica/CQRS) e 7 (API).

### 5. Deployment autonomo ★ — "questo è un test cruciale"

*"È **effettivamente possibile distribuire una modifica a un singolo servizio senza dover ridistribuire altri servizi**? Esistono **dipendenze nascoste** — come **librerie condivise che impongono aggiornamenti sincronizzati o dipendenze implicite dal database** — che interferiscono con questo processo?"*

Questo è il **test definitivo per il Distributed Monolith**. Se la risposta è no, hai microservizi solo di nome.

## 6. Autonomia del team

*"La decomposizione consente a un piccolo team di assumersi la piena responsabilità del proprio servizio — dallo sviluppo al deployment alla gestione — con dipendenza minima da altri team?"*

## 7. Allineamento al business

*"I confini dei servizi hanno senso da una prospettiva di business? Sono allineati a capability di business consolidate o a sottodomini ben definiti?"*

### La conclusione onesta

*"Non esiste una formula perfetta, e spesso si tratta di un processo iterativo. Si conduce un'analisi iniziale, la si valuta rispetto a questi criteri, e poi si affinano i confini."*

---

## Identifying and Decomposing Microservices for the E-Shop Domain

Questa è la parte **operativa**: come si passa concretamente dai requisiti ai servizi.

### Passo 1 — Partire dalle user story

I requisiti funzionali espressi come user story:

- "Come utente, voglio **visualizzare l'elenco dei prodotti**"
- "Come utente, voglio **aggiungere prodotti al carrello**"
- "Come utente, voglio **controllare il carrello**"
- "Come utente, voglio **creare un ordine**"
- "Come utente, voglio **vedere la cronologia degli ordini**"

*"Questa prospettiva **user-centrica** guida la nostra comprensione delle responsabilità principali del sistema."*

### ★ Passo 2 — La tecnica dei sostantivi e dei verbi

*"Abbiamo evidenziato alcune parole chiave con **tag rossi e neri**. Perché? Perché è una tecnica utile per definire i confini di un potenziale servizio, soprattutto pensando in ottica Domain-Driven Design."*

La regola:

| Colore | Categoria grammaticale | Cosa rappresenta nel dominio                                                                                    |
|--------|------------------------|-----------------------------------------------------------------------------------------------------------------|
| Nero   | Sostantivi (nomi)      | Le <b>entità</b> e i <b>cluster</b> fondamentali → candidati a diventare <b>sottodomini</b> / servizi           |
| Rosso  | Verbi (azioni)         | I <b>processi</b> e le <b>interazioni</b> dentro o tra i domini → candidati a diventare <b>operazioni</b> / API |

*"Consiste nell'**estrarre i sostantivi chiave** dai nostri casi d'uso e requisiti. Questi sostantivi spesso corrispondono alle **entità o ai cluster fondamentali del dominio**, e ci aiutano a iniziare a pensare a **sottodomini distinti o bounded context**."*

*"I **verbi** — come browse, add, checkout — spesso **rappresentano processi o interazioni all'interno o tra questi domini**."*

**I sostantivi estratti dal sottoinsieme iniziale: Product, Basket, Order**

**Altri sostantivi importanti emersi ampliando lo sguardo:** Customer/User, Account, Order Details, Address, Supplier, Brand, Category

**Il risultato — il diagramma delle responsabilità:**

*"Si arriva a un **diagramma delle responsabilità degli oggetti**: i **riquadri rappresentano i domini** (Basket, Order), le **linee rappresentano le azioni e le interazioni** tra questi domini."*

Esempio concreto:

- Il dominio **Basket** contiene: AddItem, GetTotalPrice
- Il dominio **Order** contiene: Checkout, GetOrderSummary

**Perché questa tecnica funziona:** il linguaggio del business è già la struttura del dominio. Le persone che lavorano in azienda hanno passato anni a raffinare un vocabolario che riflette come il business è realmente organizzato. Estrarre i sostantivi è un modo economico di far emergere quella struttura implicita.

**Passo 3 — I 4 microservizi core**

Applicando i pattern di decomposizione ai sostantivi e sottodomini identificati:

| Microservizio                    | Business Capability /<br>Sottodominio | Responsabilità                                                                                      |
|----------------------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Product Catalog</b>           | Product Management /<br>Catalog       | Tutti gli aspetti delle informazioni prodotto:<br>dettagli, categorie, eventualmente <b>ricerca</b> |
| <b>Basket</b>                    | Basket Management                     | Creazione e aggiornamento dei carrelli utente                                                       |
| <b>Ordering</b>                  | Order Management                      | <b>L'intero ciclo di vita dell'ordine:</b> creazione,<br>elaborazione pagamento, tracking           |
| <b>Identity (o<br/>Customer)</b> | User Management                       | Account utente, profili, <b>autenticazione e<br/>autorizzazione</b>                                 |

*"Questi quattro servizi rappresentano **un buon punto di partenza** per decomporre il backend dell'eShop, poiché **ognuno si concentra su un'area distinta della funzionalità di business**."*

★ **Passo 4 — L'ulteriore scomposizione di "Ordering"**

Questa è la parte più interessante, e risponde direttamente al dubbio sollevato alla fine della Lezione 5.

*"Consideriamo il processo di **order fulfillment** e il suo potenziale per un'ulteriore scomposizione."*

| Servizio candidato                                                                                                                                                                                                   | Perché merita di essere estratto                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Payment                                                                                                                                                                                                              | "È una <b>capability altamente specializzata e critica</b> . Potrebbe meritare un proprio servizio per <b>sicurezza, compliance e integrazione con diversi payment gateway</b> " |
| Inventory                                                                                                                                                                                                            | "Dopo che un ordine viene effettuato, <b>i livelli di inventario devono essere aggiornati</b> . Può essere gestito da un servizio dedicato"                                      |
| Shipping                                                                                                                                                                                                             | "Organizzare e tracciare le spedizioni è <b>un'altra capability di business specifica</b> "                                                                                      |
| Notification                                                                                                                                                                                                         | "L'invio di conferme d'ordine, aggiornamenti di spedizione ecc. <b>potrebbe essere compito di un servizio di notifica</b> "                                                      |
| "Anche se potremmo <b>iniziare con un servizio Ordering più completo</b> , dobbiamo tenere presente che <b>il processo di fulfillment sarà probabilmente ulteriormente suddiviso</b> in microservizi più specifici." |                                                                                                                                                                                  |

Nota come questo risponda al dubbio della Lezione 5 ("il Payment dovrebbe essere separato da Ordering?"): **sì, probabilmente — ma non necessariamente dal giorno uno.**

### Passo 5 — La visione a lungo termine

Allargando ulteriormente lo sguardo, altri potenziali microservizi:

**Core:** Identity, Product Catalog, Basket, Order Fulfillment

**Fulfillment:** Inventory, Payment, Shipping, Notification

**Supporto:** Marketing (promozioni e campagne), Location (se ci sono negozi fisici), Search avanzata, Rating & Review, Recommendation Engine, Finance & Reporting

"Questo illustra come un dominio complesso come l'e-commerce **possa essere suddiviso in molti potenziali microservizi**. Questo non significa che dovremmo crearli tutti dal primo giorno, ma è utile tenere a mente questa prospettiva."

### ★ Il principio conclusivo

"Il punto chiave è che questa decomposizione è **iterativa**. Si inizia con **un insieme ragionevole di servizi basati su capability di business chiare e sottodomini ben definiti**. Poi **si evolve**, applicando continuamente i pattern di decomposizione e le checklist di valutazione."

**Perché è importante:** la tentazione dell'architetto inesperto è progettare subito i 16 microservizi della visione finale. È un errore — YAGNI si applica anche qui. **Meglio 4 servizi con confini corretti che 16 con confini sbagliati.**

---

## Strangler Fig Pattern for Refactoring Monoliths into Microservices

### Il problema che risolve

"Abbiamo appena scomposto la nostra analisi iniziale in potenziali microservizi. **Ma cosa succede se si dispone già di un monolite esistente** — anche ben organizzato come quello che abbiamo progettato? **Come si procede in modo sicuro e incrementale verso questi microservizi, senza rischiare una riscrittura completa?**"



## Definizione e origine del nome

*"Il nome, coniato da Martin Fowler, si ispira alle liane del fico strangolatore (strangler fig) che crescono nelle foreste pluviali. Queste liane crescono attorno a un albero esistente, avvolgendolo gradualmente nel corso di molti anni, fino a sostituirlo completamente quando l'albero muore."*

**Definizione: Strangler Fig Pattern** — un metodo per **riscrivere e sostituire in modo incrementale** una vecchia applicazione monolitica con nuovi microservizi.

*"Invece di una riscrittura ad alto rischio e "big bang", si costruiscono gradualmente nuove funzionalità come microservizi **attorno** all'applicazione monolitica esistente."*

## Il processo (4 passi, ciclici)

### 1. Identificare la parte da sostituire

Scegliere la funzionalità del monolite che si vuole estrarre come microservizio.

### 2. Sviluppare il nuovo microservizio in modo indipendente, con il proprio database.

### 3. Introdurre lo "strangler facade" ★

*"Si introduce un **livello o un'interfaccia di routing**, spesso chiamata **strangler facade**. Può essere **parte di un API Gateway o un proxy personalizzato**. Questa interfaccia **intercetta le richieste in entrata**."*

*"Inizialmente, **la maggior parte delle richieste viene ancora instradata al vecchio monolite**, ma **per le funzionalità specifiche gestite dai nuovi microservizi**, la facade **instrada le richieste al nuovo servizio**."*

### 4. Ripetere

*"Nel tempo si identificano ulteriori funzionalità, si creano altri microservizi e si aggiorna il routing. **Gradualmente, sempre più traffico viene gestito dai nuovi microservizi**, e le funzioni nel vecchio monolite si riducono, **fino a quando il monolite viene completamente sostituito o ridotto a un nucleo molto piccolo e gestibile**."*

---

## ★ L'applicazione concreta al nostro e-shop

Questa è la parte che chiude il cerchio con la Lezione 3.

*"Ricordate la nostra applicazione backend, che internamente avevamo strutturato come **modular monolith**? **Moduli chiaramente definiti come Catalog, Basket, Ordering e Identity rappresentano un ottimo punto di partenza per applicare lo Strangler Fig**."*

Ecco perché la Lezione 3 non era un passaggio inutile: **i moduli con confini netti sono i candidati naturali all'estrazione**.

Stato iniziale

SPA → [API Service monolitico modulare] → DB condiviso

└─ Catalog

└─ Basket

└ Ordering

└ Identity

"Il nostro **primo modulo sarà Identity**. Creiamo un nuovo microservizio Identity indipendente **con il proprio database**. Forniamo una **strangler facade** — ad esempio un API Gateway. A questo punto, **le chiamate API per i dati di identity vengono instradate attraverso la facade al nuovo microservizio**, mentre **le chiamate per Catalog, Basket e Ordering continuano ad andare al monolite originale**."

## Step 2 — Strangolare il modulo Basket

└ → [Identity Service] → Identity DB

SPA → [FACADE] ———┐ → [Basket Service] → Redis ← NUOVO

└ → [API Service monolitico] → DB condiviso

└ Catalog

└ Ordering

"Una volta che il primo step è stabile, **si seleziona il modulo successivo** — ad esempio Basket — e **si ripete il processo**: si crea un nuovo microservizio Basket con una cache o database dedicato come **Redis**, e **si aggiorna la facade**."

E così via

"Si prosegue definendo un nuovo modulo, realizzandolo come microservizio e aggiornando il routing, **riducendo gradualmente le responsabilità dell'applicazione monolitica originale**."

## ★ Perché questo pattern è superiore alla riscrittura

| Aspetto                           | Big Bang Rewrite                           | Strangler Fig                                                                            |
|-----------------------------------|--------------------------------------------|------------------------------------------------------------------------------------------|
| Rischio                           | Altissimo, concentrato in un unico momento | "Riduce significativamente il rischio: <b>si apportano piccole modifiche gestibili</b> " |
| Il sistema durante la transizione | Spesso fermo o in doppio mantenimento      | " <b>Il sistema rimane operativo per tutto il periodo</b> "                              |
| Consegna di valore                | Nessuna finché non è finito tutto          | " <b>Consente una consegna continua di valore</b> "                                      |
| Reversibilità                     | Quasi impossibile tornare indietro         | Ogni step può essere invertito aggiornando il routing                                    |
| Durata                            | Mesi/anni di blackout                      | "Periodo di transizione potenzialmente lungo, <b>ma stabile</b> "                        |

"Ogni fase rappresenta una parte gestibile del lavoro, e il sistema rimane operativo per tutto il periodo."

## 📌 La nota sull'ambito del corso

"È importante chiarire l'ambito di questo corso. Lo Strangler Fig è **una strategia di migrazione cruciale** per evolvere architetture monolitiche esistenti, **ma questo corso si concentra principalmente sulla progettazione di architetture a microservizi**."

*"Pertanto, **partiamo dal presupposto che questa transizione incrementale sia stata implementata con successo** per arrivare all'architettura a microservizi progettata, e il nostro obiettivo è comprendere i microservizi nei loro vari aspetti e considerazioni verticali."*

---

## **Problem: How Microservices Communicate with Each Other**

**EVAL** → **PROBLEM** della Lezione 7.

### **Dove siamo nel percorso**

*"La costruzione di un sistema a microservizi implica diversi aspetti di progettazione. **Abbiamo completato la fase di decomposizione**, esplorando e applicando i pattern per suddividere l'applicazione. **Ora ci concentriamo sulla seconda area critica: la comunicazione** — ovvero come questi servizi distribuiti, ognuno nel proprio processo, possano comunicare tra loro **in modo efficace e affidabile**."*

### **Lo scenario che lo rende concreto**

*"Consideriamo uno scenario comune: **un utente completa il carrello e procede al checkout**. La SPA interagisce con il microservizio **Basket** per recuperare i dettagli finali del carrello e poi creare l'ordine. **Il microservizio Basket potrebbe dover comunicare informazioni al microservizio Ordering**, oppure la webapp potrebbe coordinare queste comunicazioni."*

È esattamente la chiamata sincrona `Basket` → `Ordering` introdotta deliberatamente nel codice della Lezione 5.

### ★ **Le domande aperte**

*"Come possono microservizi indipendenti comunicare tra loro attraverso una rete?"*

| # | Domanda                                                                                           | Lezione |
|---|---------------------------------------------------------------------------------------------------|---------|
| 1 | <b>Quali protocolli useranno? HTTP, gRPC o AMQP?</b>                                              | 7, 9    |
| 2 | <b>Quali formati di messaggio si scambieranno? JSON, XML, Protocol Buffers?</b>                   | 7       |
| 3 | <b>Come faranno i servizi a trovarsi a vicenda sulla rete? Cos'è la service discovery?</b>        | 7       |
| 4 | <b>Come gestiranno il versioning delle API</b> man mano che i servizi evolvono indipendentemente? | 7       |
| 5 | <b>Come progettare API efficienti</b> che evitino comunicazioni ridondanti (chatty)?              | 7, 8    |
| 6 | <b>La comunicazione dovrebbe essere sincrona o asincrona?</b> ★                                   | 7, 9    |

### **La direzione della soluzione**

*"Per rispondere a queste domande, abbiamo bisogno di **API chiaramente definite per ogni microservizio**, e dobbiamo **standardizzare i protocolli di comunicazione e i formati dei dati**. Dobbiamo esplorare **le connessioni sincrone comuni usando API RESTful basate su HTTP**."*

*"Come soluzione, studieremo **i pattern di comunicazione dei microservizi**."*